NeurIPS 2022: Emulating the Slowest Step in Lensing Cosmography with a Neural Network

SKiNN swaps a slow physics-based kinematics code for a CNN — about 300x faster, at sub-percent accuracy

How fast is the universe expanding? The question, captured by the Hubble constant (H0), sits at the center of one of the sharpest tensions in modern cosmology: the value inferred from the early universe and the value measured directly in the nearby universe disagree by more than statistics can explain. Time-delay cosmography, which uses gravitational lensing, offers an independent route to H0 that does not lean on the traditional distance ladder — which is exactly what makes it valuable.

The idea is elegant: a foreground galaxy bends the light of a background quasar into multiple images, and because those images trace different paths, their light arrives days to months apart. Match that time delay to a model of the lens galaxy's mass and you can back out a cosmic distance scale, and from it H0. The catch is degeneracy. Lensing suffers from the mass-sheet degeneracy: many different mass distributions reproduce almost the same image, and the imaging data alone cannot tell them apart. Breaking that degeneracy requires an independent constraint — the stellar kinematics of the lens galaxy, essentially how fast its stars are moving.

The problem is that kinematics modeling is slow. To solve the kinematics and the imaging self-consistently, you predict the galaxy's velocity field with a physics code like JAM (Jeans Anisotropic Modeling), and inside an MCMC sampler that code must be recomputed for every single likelihood evaluation. One call takes on the order of fifteen seconds; multiplied over the hundreds of thousands of evaluations a chain needs, jointly exploring the lensing and kinematics parameter spaces becomes computationally prohibitive, so in practice modelers fall back to fitting the two pieces separately. This paper's contribution, SKiNN (Stellar Kinematics Neural Network), targets exactly that bottleneck: train a network to imitate JAM's output, and the slowest step becomes a single forward pass.

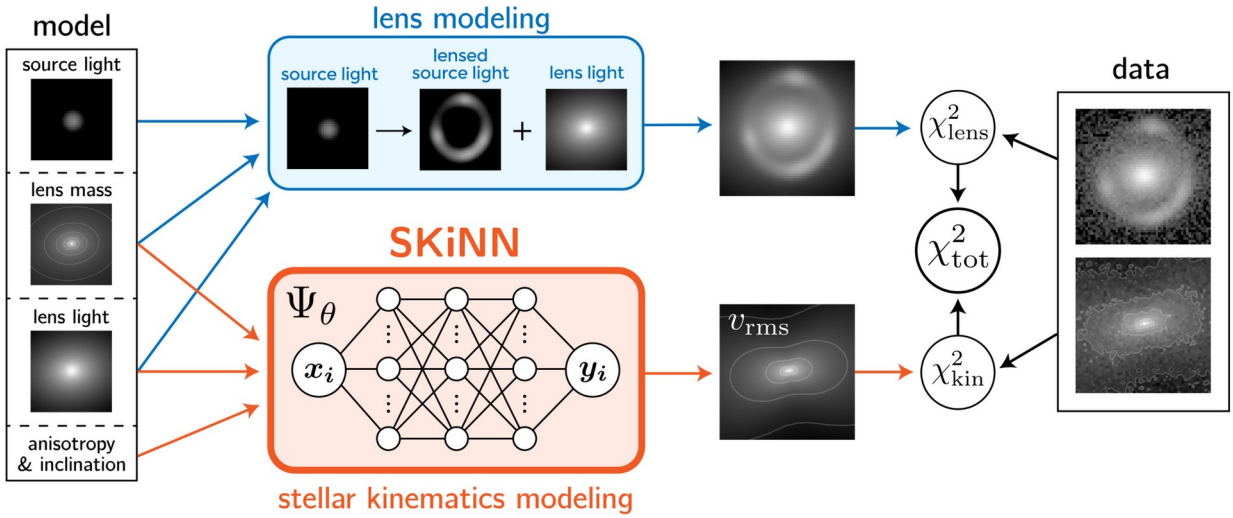

Figure 1. Where SKiNN fits in a joint modeling framework. The top row (blue) is traditional lens modeling: a model for the source, lens mass, and lens light builds an image that is compared to the imaging data, giving chi-squared_lens. The bottom row (orange) is SKiNN: lens mass, lens light, and kinematics parameters feed a network that outputs a vrms velocity image, compared to the kinematic data to give chi-squared_kin. The combined chi-squared_tot is minimized to optimize the whole model jointly. SKiNN replaces only the kinematics piece; the rest of the framework's physics is untouched.

Paper (NeurIPS 2022): https://neurips.cc/virtual/2022/57003

Code: https://github.com/mattgomer/SKiNN

Why kinematics modeling is the bottleneck

Spherical Jeans models, the traditional shortcut, are too crude for spatially resolved data and are not self-consistent with the elliptical mass models used in lensing. As instruments like JWST deliver spatially resolved kinematics of lens galaxies, the natural next step is axisymmetric modeling with a code like JAM. But JAM is expensive, and the expense compounds: it lives in the innermost loop of the likelihood evaluation, and an MCMC chain can call that loop hundreds of thousands of times. The cost is not running it once; it is running it again and again.

The key observation is that JAM, though slow, is deterministic and fully calculable. That makes it a good candidate for emulation: generate a batch of JAM outputs, and train a neural network to approximate the mapping. Training is a one-time cost; afterwards each inference is cheap. Importantly, the network does not replace the physics of the overall model — it replaces only the parameters-to-velocity-image computation, while everything around it stays physical.

How SKiNN is built

Formally, SKiNN is a function Psi-theta that maps an 8-dimensional vector of galaxy parameters to a d x d image of vrms (the quadrature sum of velocity dispersion and rotational velocity). The eight inputs describe a Power-law Elliptical Mass Distribution (PEMD) plus an elliptical Sersic light profile — six shape parameters in total — together with the galaxy's inclination i and its orbital anisotropy beta. The architecture is convolutional: five blocks, each

with two 2D convolutional layers followed by upsampling and a ReLU nonlinearity, for roughly 7.07 million trainable parameters. It is trained with a standard mean-squared-error loss and the Adam optimizer so that the network learns to reproduce JAM's output.

The training data is generated entirely by JAM: 5,000 input-output pairs, split 4,000 for training, 500 for validation, and 500 held out for testing. Each velocity image is 551 x 551 pixels, at higher resolution than real observations. A single JAM image takes about fifteen seconds to produce, which is precisely why emulation pays off. The network is implemented in PyTorch with PyTorch Lightning and trained on five Tesla P100 GPUs (16 GB each) in about a day. The parameter ranges are physically motivated rather than sampled arbitrarily.

About 300x faster, at under one percent error

Once trained, SKiNN turns a parameter vector into a velocity image in about 50 milliseconds on a single GPU, against roughly 15 seconds for JAM — about a 300x speedup (and the GPU is itself about 10x faster than a CPU here). The real question is whether that speed costs accuracy. The authors answer it by computing the per-pixel relative error on the 500 held-out test images.

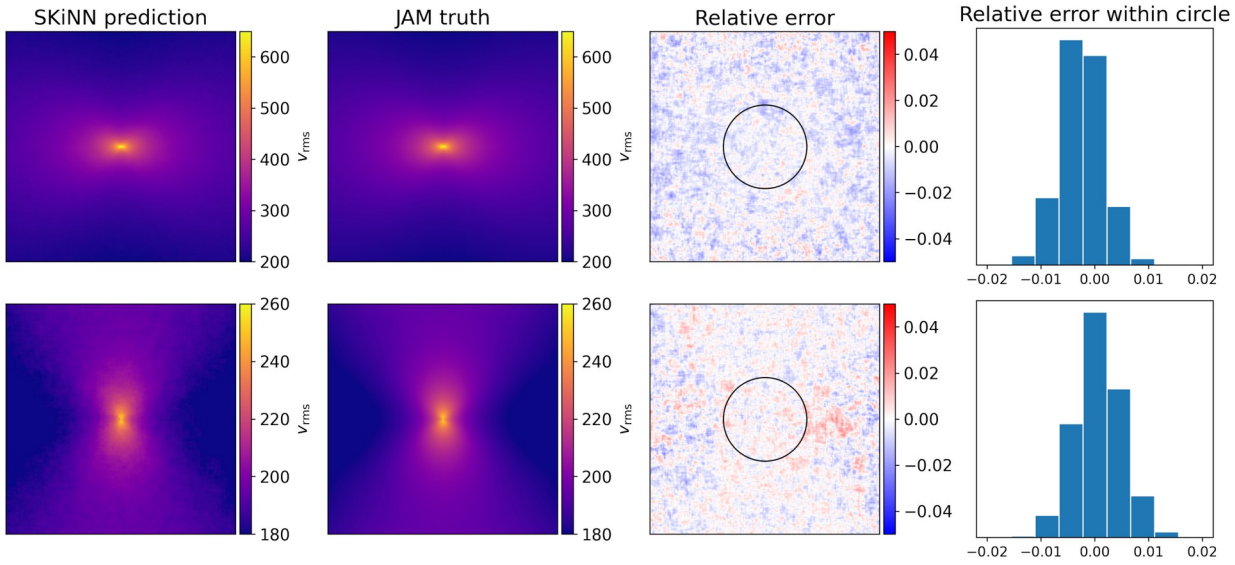

Figure 2. Two vrms images produced by SKiNN compared against JAM ground truth. The first two columns show SKiNN's prediction and the JAM truth; the third shows the per-pixel relative error, with the black circle marking the innermost two arcseconds, where real data is most constraining. Errors can reach a few percent in the outskirts, but inside the circle they stay below 1% almost everywhere; the histograms on the right confirm that the within-circle relative error concentrates within plus or minus one percent.

The reading is clear. Errors of a few percent do appear in the image outskirts, but those regions carry little constraining power anyway. In the innermost two arcseconds, where the data actually speaks, the emulation matches JAM to within plus or minus one percent for nearly every pixel. Averaged over whole images, the median absolute error across the 500 test images is about 0.47%, and the 90th percentile is about 1.1% — both comfortably below the roughly 2%-or-greater systematic uncertainty typical of real velocity measurements. In other words, the emulation error is swamped by observational noise, so it should not dominate the science.

Table 1. Speed and accuracy of SKiNN versus JAM for a single vrms image.

Metric	JAM (physics code)	SKiNN (neural network)
Time per image	about 15 s	about 50 ms (GPU)
Relative speedup	1x (baseline)	about 300x
Error within inner 2 arcsec	ground truth	< 1% almost everywhere
Whole-image error, median / 90th pct	-	about 0.47% / about 1.1%

Why it matters, and where it stops

By turning the slowest step of joint modeling into a cheap forward pass, SKiNN makes it computationally feasible, for the first time, to fit gravitational lensing and spatially resolved kinematics together — and it is precisely the kinematic constraint that corrects the largest source of uncertainty in H0 measurements. The authors are candid about the scope: this is a proof-of-concept. There is no formal ablation table; performance is characterized by the per-pixel error distribution on the test set, the emulator is only valid within the sampled parameter ranges, and the outskirts of the image do carry larger errors.

Perhaps the more transferable idea is the strategy itself: when a model contains an expensive-but-calculable component, you can train a network to emulate that one piece while keeping the surrounding physics intact. You buy the speed of a neural network without giving up the interpretability of the full physical model. That 'replace only the slowest block' pattern seems likely to carry over to many scientific modeling problems well beyond astrophysics.